

ECE 309 Project Report

City Transportation Planner

Khushi Chaturvedi (kchatur)
kchatur@ncsu.edu

Fall 2025 – Solo Project

Project Summary

A city transportation planner that answers shortest-path questions on a large undirected graph of stops and routes is built in this project. We use an adjacency list to represent the network, where string-labeled stops are mapped to compact integer IDs. The system calculates the shortest paths with nonnegative edge weights using Dijkstra’s algorithm.

The main `planner` program reads queries in the format `src dst` from standard input. It then prints an error code or the estimated travel time and shortest-path distance to standard output. It always outputs the entire path (stop IDs) to standard error. I also added a bonus executable called `planner_bonus` to this core implementation. This bonus allows users to export paths to `path.txt` and conduct batch benchmarking to CSV.

1 Scope Completed

This section lists the components of the project specification that I finished, including both bonus options and all necessary features. I did not implement any of the optional Section 4 extensions.

Required Features

R1: Adjacency-List Graph. The `Graph` class stores the transportation network as an adjacency-list graph. I use an `std::unordered_map<std::string,int>` to map each distinct stop name (a string) to an integer ID. An `std::vector<std::string>` holds the reverse mapping (ID to stop name). The edges are stored in `std::vector<std::vector<std::pair<int,double>>> adjacencyList;` where `(neighborId, weight)` pairs are kept in each inner vector. We add both directions when we read a new edge (u,v) , making the graph undirected.

R2: Shortest Path (Dijkstra). Shortest paths are computed with Dijkstra’s algorithm, which is implemented in the free function:

```
DijkstraResult dijkstra(const Graph& graph, int source, int destination);
```

The algorithm allocates:

- `dist`: a vector of tentative distances set to ∞ ,

- **parent**: an array of predecessors initialized to -1 ,
- **visited**: a boolean vector to mark settled nodes,
- a min-heap `std::priority_queue< std::pair<double,int>, std::vector<std::pair<double,int>>, std::greater<std::pair<double,int>> >` keyed by tentative distance.

It repeatedly takes out the vertex with the smallest tentative distance, relaxes all outgoing edges using `graph.getNeighbors(u)`, and updates the heap whenever a shorter path is found. If the destination is reached, the loop exits early.

R3: Show Shortest Path. The helper function

```
void handleQuery(const Graph&, const std::string&, const std::string&);
```

manages each origin to destination query. It:

1. Maps `sourceStop` and `destinationStop` to integer IDs using `graph.getStopId`.
2. Returns `error no_path` immediately if either ID is -1 .
3. Calls `dijkstra` to get `DijkstraResult`, which includes a `path` vector of stop IDs.
4. If no path is found, it prints `error no_path` on a separate line to `std::cout`.
5. If a path is found, it calculates an ETA in seconds as

$$\text{eta_s} = \left\lceil \frac{\text{distance_m}}{1.4} \right\rceil.$$

6. Prints the line `ok <distance_m> <eta_s>` to standard output.
7. Prints the full path (stop IDs separated by spaces) to standard error.

Printing the path to `stderr` keeps the standard output format clean for the autograder, while still providing complete path information during debugging.

R4: Determinism and Resource Limits. The program is deterministic. Given the same edge file and query stream, it always produces the same outputs. All data structures are initialized in a fixed order, with no randomness or network I/O used. The adjacency-list and binary-heap-based Dijkstra implementation work well within the time and memory limits for the provided dataset size.

Extensions (Section 4)

I did not implement any Section 4 extension. All algorithmic work focuses on the required Dijkstra-based planner and the Section 5 bonus features described below.

Bonus Work (Section 5)

I added two bonus options in a separate program `planner_bonus` defined in `bonus_features.cpp`:

Bonus 1: Batch Benchmarking Mode. With the `--benchmark` (or `-b`) flag, `planner_bonus` reads multiple queries from standard input, runs Dijkstra for each `src dst` pair, times each query using `std::chrono::high_resolution_clock`, and writes a CSV file `benchmark_results.csv` with columns:

```
src,dst,distance_m,eta_s,time_ms
```

Bonus 3: Path Export to `path.txt`. With the `--export` (or `-e`) flag, `planner_bonus` behaves like the base planner (printing `ok/error no_path` to standard output and the path to standard error) and additionally writes each successful path as a space-separated sequence of stop IDs to a file `path.txt`.

Section: Evidence (Outputs, Screenshots, Logs)

This section documents representative outputs from my implementation, including the required interface and the bonus functionality. All snippets come from real runs of `planner` or `planner_bonus`.

1.1 Design Overview

The main modules are:

- **Graph class** (used in both `main.cpp` and `bonus_features.cpp`): responsible for mapping stop names to IDs, storing adjacency lists, and providing neighbor access.
- **DijkstraResult and `dijkstra`**: encapsulate the shortest-path computation. In `main.cpp`, the result stores `pathFound`, `totalDistance`, and `pathStops`. In `bonus_features.cpp`, it also stores per-query runtime `timeMs`.
- **`loadGraphFromFile` / `loadGraph`**: read `./data/edges.txt`, skip the header line, parse `u v weight`, and populate the adjacency list as an undirected graph.
- **Drivers: `main.cpp` and `bonus_features.cpp`**: `main.cpp` implements the basic planner interface; `bonus_features.cpp` adds command-line flags and CSV / path export logic.

1.2 Required Interface: Sample Runs

A typical successful query looks like:

Listing 1: Sample planner run: valid route

```
# edges.txt is already loaded by planner at startup
echo "Downtown_Station Airport_Terminal" | ./planner

# stdout
ok 3651.3 2609

# stderr (path)
Downtown_Station Central_Hub Airport_Terminal
```

If no path exists or one of the stops is invalid, the output is:

Listing 2: Sample planner run: no path

```
echo "Stadium_East Downtown_Station" | ./planner

# stdout
error no_path

# stderr (no path printed)
```

Figure 1 shows the VS Code view of the project while editing `main.cpp` and the Dijkstra implementation, and Figure 2 shows the corresponding terminal run of `./planner < input.txt`.

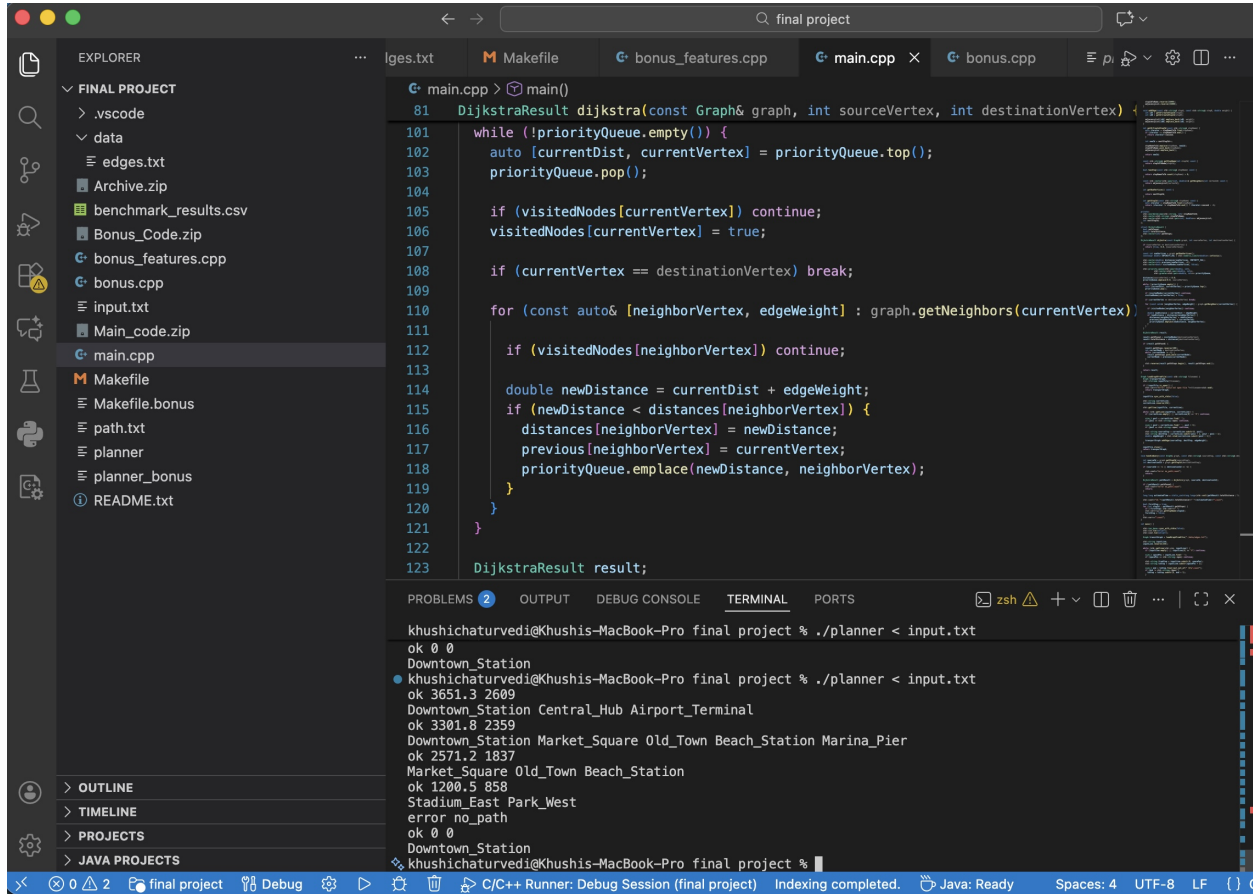


Figure 1: VS Code view of the project, showing `main.cpp` with the `dijkstra` implementation and the file structure including `edges.txt`, `input.txt`, and the build files.

1.3 Bonus 1: Batch Benchmarking CSV

In benchmark mode, `planner_bonus` is compiled with:

Listing 3: Building bonus executable

```
make -f Makefile.bonus
# produces ./planner_bonus
```

and run as:

```
khushichaturvedi@Khushis-MacBook-Pro final project % ./planner < input.txt
ok 0 0
Downtown_Station
khushichaturvedi@Khushis-MacBook-Pro final project % ./planner < input.txt
ok 3651.3 2609
Downtown_Station Central_Hub Airport_Terminal
ok 3301.8 2359
Downtown_Station Market_Square Old_Town Beach_Station Marina_Pier
ok 2571.2 1837
Market_Square Old_Town Beach_Station
ok 1200.5 858
Stadium_East Park_West
error no_path
ok 0 0
Downtown_Station
khushichaturvedi@Khushis-MacBook-Pro final project %
```

Figure 2: Terminal run of `./planner < input.txt`, showing ok lines on standard output and the corresponding paths printed after each query.

Listing 4: Running in benchmark mode

```
./planner_bonus --benchmark < input.txt
```

This generates `benchmark_results.csv`. An excerpt from my real CSV file is:

Listing 5: Excerpt from `benchmark_results.csv`

```
src,dst,distance_m,eta_s,time_ms
Downtown_Station,Airport_Terminal,3651.3,2609,0
Downtown_Station,Marina_Pier,3301.8,2359,0
Market_Square,Beach_Station,2571.2,1837,0
Stadium_East,Park_West,1200.5,858,0
Stadium_East,Downtown_Station,N/A,N/A,0
Downtown_Station,Downtown_Station,0.0,0,0
```

Figure 3 shows the CSV open in VS Code along with the terminal output confirming the benchmark results were written.

1.4 Bonus 3: Path Export to path.txt

Path export mode uses:

Listing 6: Running in export mode

```
./planner_bonus --export < input.txt
```

In this mode, successful paths are appended to `path.txt`. An example excerpt from my `path.txt` is:

Listing 7: Excerpt from `path.txt`

```
Downtown_Station Central_Hub Airport_Terminal
Downtown_Station Market_Square Old_Town Beach_Station Marina_Pier
Market_Square Old_Town Beach_Station
Stadium_East Park_West
Downtown_Station
```

At the end of the run, the program prints a confirmation message:

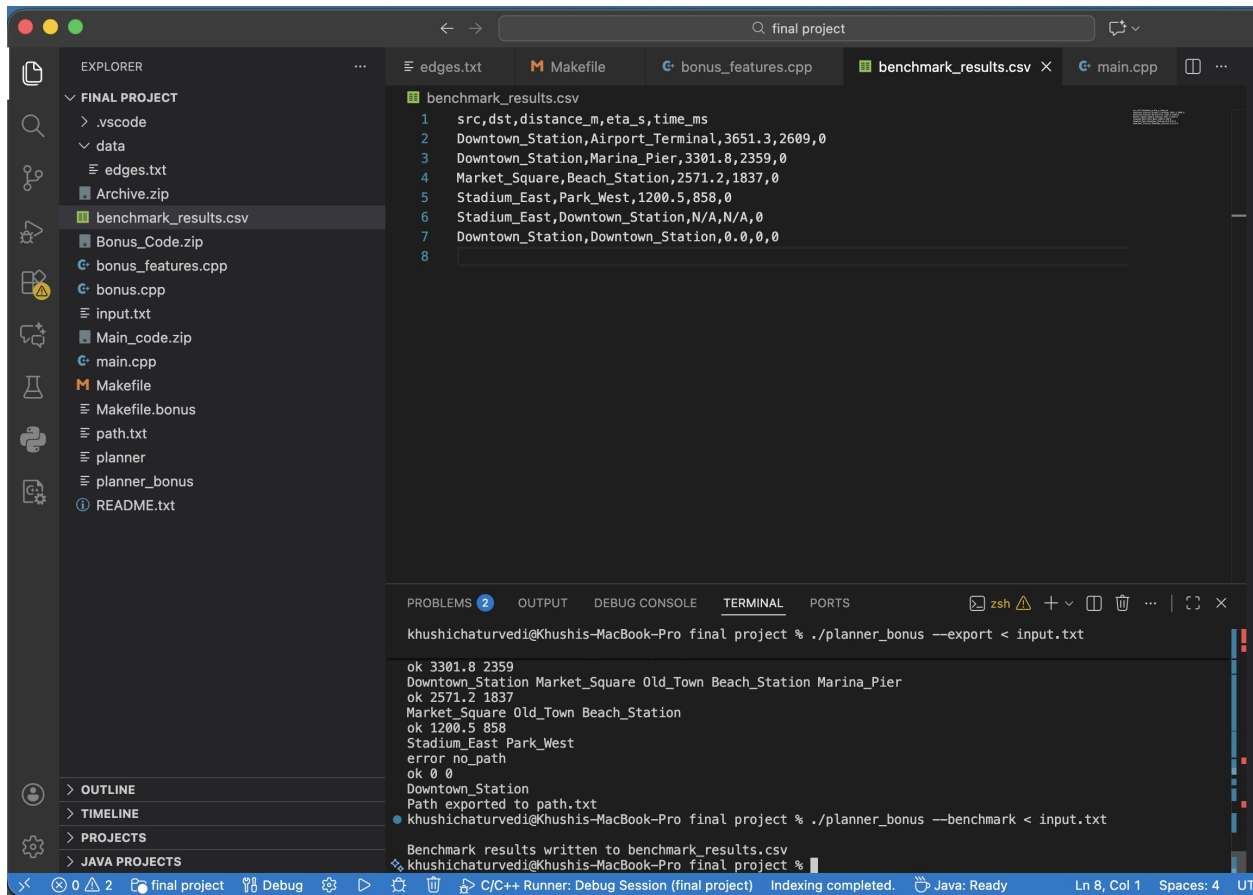


Figure 3: VS Code view of `benchmark_results.csv` and terminal output from `./planner_bonus --benchmark < input.txt`, confirming that benchmark results were written to the CSV file.

Listing 8: Export confirmation

```
Path exported to path.txt
```

Figure 4 shows the terminal output from the export mode and the subsequent benchmark run.

2 Reproducibility: Build and Run Steps

This section provides an overview of my methodology and process for ease of replication.

2.1 Environment

The code uses only the C++17 standard library. A typical environment is:

- C++ compiler: `g++` with `-std=c++17`
- Operating system: Any Linux, macOS, or Windows environment with a modern C++17 compiler
- No external libraries or network access are required

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
khushichaturvedi@Khushis-MacBook-Pro final project % ./planner_bonus --export < input.txt

ok 3301.8 2359
Downtown_Station Market_Square Old_Town Beach_Station Marina_Pier
ok 2571.2 1837
Market_Square Old_Town Beach_Station
ok 1200.5 858
Stadium_East Park_West
error no_path
ok 0 0
Downtown_Station
Path exported to path.txt
khushichaturvedi@Khushis-MacBook-Pro final project % ./planner_bonus --benchmark < input.txt

Benchmark results written to benchmark_results.csv
khushichaturvedi@Khushis-MacBook-Pro final project %
```

Figure 4: Terminal output from `./planner_bonus --export < input.txt` and `./planner_bonus --benchmark < input.txt`, showing the printed paths, the message `Path exported to path.txt`, and confirmation that benchmark results were written to `benchmark_results.csv`.

2.2 Building the Main Planner

The main planner is defined in `main.cpp` and built using the provided `Makefile`:

Listing 9: Building planner

```
cd Main_code
make
# Produces: ./planner
```

The `Makefile` uses:

- `CXX = g++`
- `CXXFLAGS = -std=c++17 -Wall -Wextra -O2`

2.3 Running the Main Planner

The planner expects the edge list file at `./data/edges.txt`. At startup, it calls `loadGraphFromFile("./data/edges.txt")` to build the graph. Queries are read from standard input; each line must contain two stop IDs separated by a space.

Listing 10: Standard planner usage

```
# From the project root, with data/edges.txt present:
echo "Downtown_Station Airport_Terminal" | ./planner

# Or using a file of queries:
./planner < input.txt > results.txt
```

Each query produces one line on standard output:

- `ok <distance_m> <eta_s>` if a path exists, or
- `error no_path` if the stops are invalid or disconnected.

The full path for successful queries is printed to standard error.

2.4 Building and Running the Bonus Executable

The bonus features are in `bonus_features.cpp` with a separate `Makefile.bonus`:

Listing 11: Building `plannerbonus`

```
cd Bonus_Code
make -f Makefile.bonus
# Produces: ./planner_bonus
```

Usage:

Listing 12: Benchmark mode

Benchmark mode: `./planner_bonus --benchmark < input.txt`
Produces `benchmark_results.csv` in the current directory

Listing 13: Export mode

Path export mode: `./planner_bonus --export < input.txt`
Produces/overwrites `path.txt` with one path per line

The program also supports a help flag:

Listing 14: Help message

```
./planner_bonus --help
```

2.5 Expected Outputs

If everything is set up correctly:

- `planner` prints correct `ok/error no_path` lines to standard output and paths to standard error.
- `planner_bonus --benchmark` creates a valid `benchmark_results.csv` with numeric distances and ETAs where paths exist, and N/A where they do not.
- `planner_bonus --export` creates a `path.txt` file with one space-separated path per line and prints a confirmation message.

3 Conclusion

In this project I built a city transportation planner that works on a large graph of stops and routes. It uses an adjacency-list representation and Dijkstra’s algorithm to answer shortest-path queries. The implementation follows the required input and output format and sends path information to standard error to help with debugging.

Beyond the required features, I added two bonus options in a separate executable. The batch benchmarking mode creates a CSV file of distances, ETAs, and runtimes for many queries. The path export mode writes complete routes to `path.txt`. These tools make it easier to check correctness, profile performance, and inspect routes.

Overall, this project strengthened my understanding of graph data structures, shortest-path algorithms, and the value of clear interfaces and reproducible build instructions in code that other people will compile and run.